

Algorithms and Data Structures II.
Written Exam – Example Questions 2024

Rules for the Exams

Dear Student,

You get theoretical questions and other questions about illustrating algorithms learned this semester. Most of these algorithms have different variants. You must present them as you have seen them in the classroom.

Write the structure diagrams and definitions, and illustrate each algorithm the way you learned in my lessons. You have patterns for illustration in Teams.

For each exercise about illustrating algorithms, follow the following process:

- Carefully copy the input data given in the exercises.
- You must use the data given in your exercise.
- Illustration on another input will not be accepted.
- You draw your illustrations on some sheets of originally blank paper or the paper containing the exercise.

Best wishes, Tibor Asvanyi

DC Data Compression

DC.1 The Huffman coding

<http://en.wikipedia.org/wiki/Huffman_coding>

DC.1.1 We would like to compress the text

MATEKFELELETEMKETTESLETT

with Huffman coding. Draw the Huffman tree, give the Huffman code of each letter, the Huffman code of the text, and the length of the Huffman code in bits. Show the letters of the original text in the Huffman code of it.

DC.1.2 Solve Exercise DC.1.1 with the following text.

EMESE MAI SMSE NEM NAIV MESE

DC.1.3 We have compressed a text with Huffman coding. Given an internal node of the Huffman tree, number 0 labels the edge, which goes to its child with a smaller frequency value, and number 1 labels the edge, which goes to its child with a higher frequency value. The frequencies of the different characters are the following ones. R:2, T:4, O:5, G:7, E:13, and A:17. The beginning of the compressed code is

0111 1100 0100 1101 0

without spaces. (Blanks are included here only to increase readability.) Draw the Huffman tree and decompress the code above. What are the codes of the characters above? What is the decompressed text?

DC.2 The Lempel–Ziv–Welch (LZW) algorithm

<<http://en.wikipedia.org/wiki/Lempel-Ziv-Welch>>

DC.2.1 We have compressed a text with the Lempel-Ziv-Welch algorithm. The text contains the letters 'A', 'B', and 'C'. Their codes are 1, 2, and 3, respectively. While building the dictionary, the first unused positive integer was selected for the code of each new word. In this way, we have received the following code.

1 2 4 3 5 6 9 7 1

Give the original text and the complete dictionary.

DC.2.2 Solve Exercise DC.2.1 with the following LZW code.

1 2 4 3 5 8 1 10 11 1

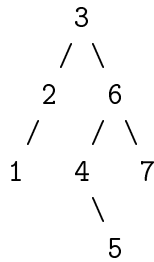
DC.2.3 Illustrate LZW compression on text *CBABABABCBABABAB*.

DC.2.4 Solve Exercise DC.2.1 with the following LZW code.

1 2 3 4 6 5 9 7 11

AVL trees

Note: $\{ [(1) 2 ()] 3 [(\langle \rangle 4 \langle 5 \rangle) 6 (7)] \}$ is the parenthesised form of the following tree.



AVL.1 Given an initially empty AVL tree t . Illustrate the insertion of numbers 1, 2, 7, 3, 5, 6, 8, 9, and 4 into it in the given order. Show the balances of each node on your illustrations. When you balance a subtree, make sure which subtree is balanced.

AVL.2 Given the AVL tree $t = \{ [(1) 2 ()] 3 [(\langle \rangle 7 \langle 9 \rangle) 10 (11)] \}$. Draw the tree with its balances. Illustrate the deletion of the numbers 2, 1, and 7 from t in the given order. Show the balances of each node on your illustrations. When you balance a subtree, make sure which subtree is balanced.

AVL.3 Given the AVL tree $t = \{ [(1) 2 ()] 3 [(\langle \rangle 7 \langle 9 \rangle) 10 (11)] \}$. Draw the tree with its balances. Illustrate the removal of its minimal and maximal key, **in each case on the original tree**. Show the balances of each node on your illustrations. When you balance a subtree, make sure which subtree is balanced.

AVL.4 Given the AVL tree $t = \{ [(2) 4 (\langle 6 \rangle 8 \langle 10 \rangle)] 12 [() 14 (16)] \}$. Draw the tree with its balances. Illustrate the insertion of keys 1, 5, and 15, **in each case on the original tree**. Show the balances of each node on your illustrations. When you balance a subtree, make sure which subtree is balanced.

AVL.5 Given the $t = \{ [(2) 4 ()] 6 [(\langle 8 \rangle 10 \langle 12 \rangle) 14 (16)] \}$ AVL tree. Draw the tree with its balances. Illustrate the removal of its minimal and maximal key, **in each case on the original tree**. Show the balances of each node on your illustrations. When you balance a subtree, make sure which subtree is balanced.

AVL.6 We already know BSTs. Present the definitions characterising AVL trees.

AVL.7 Define Fibonacci trees. What do you know about the size of a Fibonacci tree with height h ? Why are Fibonacci trees essential?

AVL.8 What theorem do you know about the height of a balanced binary search tree? Why this theorem is essential?

AVL.9 What theorem do you know about the height of a balanced binary search tree? Present the outline of its proof.

AVL.10 What kind of balancing rotations do you know? Draw the four general rotation schemes which are essential during insertion.

AVL.11 Draw the two general rotation schemes which are not used during insertion. Why are these important?

AVL.12 Draw the structure diagram of procedure `AVLinsert()` as we learned in the lecture. (Do not draw the structure diagrams of its subroutines.)

AVL.13 As we learned in the lecture, draw the structure diagram of procedure `leftSubTreeGrown()`. (Do not draw the structure diagram of its subroutines.)

AVL.14 As we learned in the lecture, draw the structure diagram of procedure `rightSubTreeGrown()`. (Do not draw the structure diagrams of its subroutines.)

AVL.15 As we learned in the lecture, draw the structure diagram of balancing rotation $(++,+)$.

AVL.16 As we learned in the lecture, draw the structure diagram of balancing rotation $(++, -)$.

AVL.17 As we learned in the lecture, draw the structure diagram of balancing rotation $(++,0)$.

AVL.18 As we learned in the lecture, draw the structure diagram of balancing rotation $(--, +)$.

AVL.19 As we learned in the lecture, draw the structure diagram of balancing rotation $(--, -)$.

AVL.20 Draw the structure diagram of balancing rotation $(--,0)$.

AVL.21 Draw the structure diagrams of procedures AVLremMin() and leftSubTreeShrunk() as we learned in the lecture. (Do not draw the structure diagrams of their other subroutines.)

AVL.22 As we learned in the lecture, draw the structure diagram of procedure rotate_PP(). (Do not draw the structure diagrams of its subroutines.)

AVL.23 Draw the structure diagram of procedure AVLdel() as we learned in the lecture. (Do not draw the structure diagrams of its subroutines.)

AVL.23 As we learned in the lecture, draw the structure diagrams of procedures AVLdelRoot() and rightSubTreeMinToRoot(). (Do not draw the structure diagrams of their other subroutines.)

B+ trees

B+a, What does it mean that a B+ tree has degree d ? What is the minimal degree of a B+ tree? When do we use B+ trees instead of balanced binary search trees (like AVL trees), and why?

B+b, Present the invariant properties of the B+ trees.

B+c, What do you know about the operational complexity of the search, insert and delete operations of B+ trees? Explain your statements.

Note for the subsequent exercises:

- The general schemes of the textual representation of a non-leaf B+ tree of degree 4 are $(T1\ k1\ T2)$, $(T1\ k1\ T2\ k2\ T3)$, and $(T1\ k1\ T2\ k2\ T3\ k3\ T4)$ where T_i is the i th subtree, and k_i is a split key between the i th and the $(i+1)$ subtree.
- The general schemes of the textual representation of a leaf of a B+ tree of degree 4 are $(k1\ k2)$, and $(k1\ k2\ k3)$ where k_i is a key of the set represented with the B+ tree.
- In the example trees, we vary three kinds of pairs of brackets: $()$, $[]$, and $\{\}$ depending on the level of a subtree.

B+1 Given the following B+ tree of degree 4.

$\{ [(1\ 2\ 4)\ 5\ (6\ 8)]\ 10 [(10\ 12\ 14)\ 15\ (16\ 18)\ 20\ (20\ 22)\ 24\ (24\ 26)] \}$

Draw the tree. Illustrate the insertion of keys 5, 3, and 11, **in each case on the original tree**.

B+2 Given the following B+ tree of degree 4.

$\{ [(2\ 4)\ 5\ (6\ 8)]\ 10 [(10\ 12\ 14)\ 15\ (16\ 18)\ 20\ (20\ 22)] \}$

Draw the tree. Illustrate the deletion of keys 18, 22, and 2, **in each case on the original tree**.

B+3 Given the following B+ tree of degree 4.

$\{ [(2\ 3\ 4)\ 5\ (6\ 7)]\ 16 [(16\ 18)\ 19\ (20\ 22)] \}$

Draw the tree. Illustrate the insertion of key 1 and the deletion of key 16, **in each case on the original tree**.

B+4 Given the following B+ tree of degree 4.

$[(9\ 10)\ 11\ (12\ 13\ 14)\ 15\ (15\ 16)\ 18\ (19\ 20)]$

Draw the tree. Illustrate the insertion of key 11, then deleting keys 9 and 19, **in each case on the original tree**.

B+5 Given the following B+ tree of degree 4.

[(9 10) 11 (12 13) 14 (15 16 17) 18 (19 20)]

Draw the tree. Illustrate the insertion of key 14, then the deletion of keys 10 and 20, **in each case on the original tree**.

B+6 Given the following B+ tree of degree 4.

{ [(1 2) 3 (4 5)] 7 [(11 15 20) 27 (27 30)] }

Draw the tree. Illustrate the insertion of key 18, then deleting keys 30 and 4, **in each case on the original tree**.

32 String Matching

32.0-1 Describe the string-matching problem. Provide the necessary definitions (alphabet, pattern, text, possible shift, valid shift).

32.1 The naive string-matching algorithm

32.1-a, Present the structure diagrams of the naive string matching (Brute-Force) algorithm. What do you know about its efficiency? Why do we need more sophisticated string-matching algorithms?

32.1-1 Show the comparisons the naive string matching (Brute-Force) algorithm makes for the pattern $P = 0001$ in the text $T = 000010001010001$.

32.1-2 Suppose that all characters in the pattern P are different. Show how to accelerate the naive string matching (Brute-Force) algorithm to run in time $O(n)$ on an n -character text T .

32-x The Quick Search algorithm

<[http://aszt.inf.elte.hu/~asvanyi/ds/Quick Searching. ppt](http://aszt.inf.elte.hu/~asvanyi/ds/Quick%20Searching.ppt)>

32-x-a, Present the structure diagrams of the Quick Search algorithm. What do you know about its efficiency? What do you know about its advantages and disadvantages compared to the KMP algorithm?

32-x-1 Compute $shift[0..1]$ for the pattern $P = 000$. Show the comparisons the Quick Search string matcher makes for this pattern in the text $T = 000010001010001$.

32-x-2 Compute $shift['A'..'F']$ for the pattern $P = ABABACD$. Show the comparisons the Quick Search string matcher makes for this pattern. in the text $T = BABAEABABAFDBABABACD$.

32-x-3 Compute $shift['A','C','G','T']$ for the pattern $P = GCAGAGAG$. Show the comparisons the Quick Search string matcher makes for this pattern in the text $T = GCATCGCAGAGAGTATACAGTACG$.

32.4 The Knuth-Morris-Pratt (KMP) algorithm

32.4-a, Define the π function. What are the two properties of the π function that we learned? Explain its role in the KMP algorithm. (What kind of “minimal shift” is related to it?)

32.4-b, Present the structure diagrams of the KMP algorithm. What do you know about its efficiency? What do you know about its advantages and disadvantages compared to the Quick Search algorithm?

32.4-1 Compute $\pi[1..19]$ (also called *prefix* or π function) for the pattern *ababbabbabbabbabb*.

32.4.2a Compute $\pi[1..4]$ for the pattern $P = 0001$. Show the comparisons the Knuth-Morris-Pratt string matcher makes for this pattern in the text $T = 000010001010001$. What are the valid shifts of P on T ?

32.4.2b Compute $\pi[1..5]$ for the pattern $P = abaab$. Show the comparisons the Knuth-Morris-Pratt string matcher makes for this pattern in the text $T = aaababaabaababaab$. What are the valid shifts of P on T ?

32.4.2c Compute $\pi[1..6]$ for the pattern $P = aabaab$. Show the comparisons the Knuth-Morris-Pratt string matcher makes for this pattern in the text $T = aaabaabaabaababaab$. What are the valid shifts of P on T ?

32.4.2d Compute $\pi[1..6]$ for the pattern $P = babbab$. Show the comparisons the Knuth-Morris-Pratt string matcher makes for this pattern in the text $T = ababbabbabbabbabbabb$. What are the valid shifts of P on T ?

32.4.2e Compute $\pi[1..7]$ for the pattern $P = ABABAKI$. Show the comparisons the Knuth-Morris-Pratt string matcher makes for this pattern in the text $T = BABALABABATIBABABAKI$.

DP Dynamic programming

DP-a, Introduce *dynamic programming* compared to the *divide and conquer method*. Present an elementary example.

DP-TC Transitive closure of a graph

DP-TC-a, What is the transitive closure of a graph? What is the definition of the $T^{(k)}$ matrices? How is the transitive closure computed with these matrices? Present their known properties.

DP-TC-b, Present the structure diagram of Warshall's transitive closure algorithm. Why a single T matrix is enough instead of the matrices $T^{(0)}, \dots, T^{(n)}$? What do you know about the efficiency of the algorithm?

DP-TC-example, Present Warshall's transitive closure algorithm on the unweighted, directed graph below. Show the matrices $T^{(0)}, \dots, T^{(4)}$.

	1	2	3	4
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
4	1	0	0	0

DP-TC-BFS, How can the *transitive closure* of a graph be computed with the *Breadth-First Search* (BFS) graph algorithm? Compare the efficiency of this later algorithm with that of Warshall's transitive-closure algorithm.

DP-FW The Floyd-Warshall algorithm

DP-FW-a, What problem is solved with the Floyd-Warshall algorithm? When can this problem be solved? What is the definition of the $(D^{(k)}, \pi^{(k)})$ matrix-pairs? How is the transitive closure computed with these matrix pairs? Present their known properties.

DP-TC-b, Present the structure diagram of the Floyd-Warshall algorithm. Why a single (D, π) matrix-pair is enough instead of the matrix-pairs $(D^{(0)}, \pi^{(0)}), \dots, (D^{(n)}, \pi^{(n)})$? What do you know about the efficiency of the algorithm?

DP-FW-example, Illustrate the Floyd-Warshall algorithm on the weighted, directed graph below. Show the matrix-pairs $(D^{(0)}, \pi^{(0)}), \dots, (D^{(4)}, \pi^{(4)})$. Draw the four shortest-path trees corresponding to four lines of the final matrix pair.

	1	2	3	4
1	0	5	3	1
2	5	0	1	∞
3	3	1	0	1
4	1	∞	1	0

DP-FW-BFS, In which case can the *All-Pairs Shortest Paths Problem* be solved with the *Breadth-First Search* (BFS) graph algorithm? How can it be solved in this case? Compare the efficiency of this latter algorithm with that of the Floyd-Warshall algorithm.

DP-FW-DAG, In which case can the *All-Pairs Shortest Paths Problem* be solved with the *DAG Shortest Paths* graph algorithm? How can it be solved in this case? Compare the efficiency of this latter algorithm with that of the Floyd-Warshall algorithm.

DP-FW-DAG, In which case can the *All-Pairs Shortest Paths Problem* be solved with the *Dijkstra* algorithm? How can it be solved in this case? Compare the efficiency of this latter algorithm with that of the Floyd-Warshall algorithm.

DP-FW-QBF, In which case can the *All-Pairs Shortest Paths Problem* be solved with the *Queue-Based Bellman-Ford* algorithm? How can it be solved in this case? Compare the efficiency of this latter algorithm with that of the Floyd-Warshall algorithm.